Threat Intelligence Aggregator

Step 1 — Install Python and Prepare Your Environment

Step 2 — Understand What Threat Intelligence Is

Step 3 — Create Your First Python File Inside

the main folder create:

```
main.py
```

This file will control the entire system.

Add this first test code:

```python
print("Threat Intelligence Aggregator Started")
```

Run it.

python main.py

Step 4 — Add Sample Threat Feeds Create

files inside the **feeds folder**.

ip_feed.txt

```
192.168.1.10
8.8.8.8
45.23.12.90
```

domain_feed.txt

```
malicious.com
phishingsite.net
badactor.org
```

url_feed.txt

```
http://malicious.com/malware.exe
https://phishingsite.net/login
```

These simulate **real threat intelligence feeds**.

Step 5 — Connect Feed Loader to Main Program

File: utils/file_loader.py Code:

file_loader.py - C:/Users/asus/OneDrive/Desktop/Threat_Intelligence_Project/utils/file_loader...

File Edit Format Run Options Window Help

```python
def load_file(path):
    with open(path, "r") as file:
        data = file.read()
    return data
```

main.py - Code:

main.py - C:/Users/asus/OneDrive/Desktop/Threat_Intelligence_Project/main.py (3.14.3)

File Edit Format Run Options Window Help

```python
print("Threat Intelligence Aggregator Started")

from utils.file_loader import load_file

ip_data = load_file("feeds/ip_feed.txt")
domain_data = load_file("feeds/domain_feed.txt")
url_data = load_file("feeds/url_feed.txt")

print("IP Feed Data:\n", ip_data)
print("Domain Feed Data:\n", domain_data)
print("URL Feed Data:\n", url_data)
```

What this does:

```
Open file
↓
Read content
↓
Return data to program
```

Run: python main.py

Output

IDLE Shell 3.14.3

File Edit Shell Debug Options Window Help

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/asus/OneDrive/Desktop/Threat_Intelligence_Project/main.py =
Threat Intelligence Aggregator Started
IP Feed Data:
 # DstIP
162.243.103.246
178.62.3.223
27.133.154.218
34.204.119.63
50.16.16.211
# END 5 entries
Domain Feed Data:
 malicious.com
phishingsite.net
badactor.org

URL Feed Data:
 http://malicious.com/malware.exe
https://phishingsite.net/login
```

Step 6— Build IOC Parser (Extract Data)

File: parsers/feed_parser.py **Code:**

feed_parser.py - C:/Users/asus/OneDrive/Desktop/Threat_Intelligence_Project/parsers/feed_...

File Edit Format Run Options Window Help

```python
import re

def extract_ips(text):
    pattern = r"\b(?:[0-9]{1,3}\.){3}[0-9]{1,3}\b"
    return re.findall(pattern, text)

def extract_domains(text):
    pattern = r"\b[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}\b"
    return re.findall(pattern, text)

def extract_urls(text):
    pattern = r"https?://[^\s]+"
    return re.findall(pattern, text)
```

main.py Code:

main.py - C:/Users/asus/OneDrive/Desktop/Threat_Intelligence_Project/main.py (3.14.3)

File Edit Format Run Options Window Help

```python
print("Threat Intelligence Aggregator Started")

from utils.file_loader import load_file

ip_data = load_file("feeds/ip_feed.txt")
domain_data = load_file("feeds/domain_feed.txt")
url_data = load_file("feeds/url_feed.txt")

print("IP Feed Data:\n", ip_data)
print("Domain Feed Data:\n", domain_data)
print("URL Feed Data:\n", url_data)

from parsers.feed_parser import extract_ips, extract_domains, extract_urls

ips = extract_ips(ip_data)
domains = extract_domains(domain_data)
urls = extract_urls(url_data)

print("Extracted IPs:", ips)
print("Extracted Domains:", domains)
print("Extracted URLs:", urls)
```

Output:

```
== RESTART: C:/Users/asus/OneDrive/Desktop/Threat_Intelligence_Project/main.py =
Threat Intelligence Aggregator Started
IP Feed Data:
 # DstIP
162.243.103.246
178.62.3.223
27.133.154.218
34.204.119.63
50.16.16.211
# END 5 entries
Domain Feed Data:
 malicious.com
phishingsite.net
badactor.org

URL Feed Data:
 http://malicious.com/malware.exe
https://phishingsite.net/login

Extracted IPs: ['162.243.103.246', '178.62.3.223', '27.133.154.218', '34.204.119.63', '50.16.16.211']
Extracted Domains: ['malicious.com', 'phishingsite.net', 'badactor.org']
Extracted URLs: ['http://malicious.com/malware.exe', 'https://phishingsite.net/login']
>>>
```

Step 7 — Validate Indicators (Clean Data)

File: core/validator.py

validator.py - C:/Users/asus/OneDrive/Desktop/Threat_Intelligence_Project/core/validator.py...

File Edit Format Run Options Window Help

```python
import ipaddress

def validate_ip(ip):
    try:
        ipaddress.ip_address(ip)
        return True
    except:
        return False

def validate_domain(domain):
    return "." in domain

def validate_url(url):
    return url.startswith("http")
```

Update main.py

main.py - C:/Users/asus/OneDrive/Desktop/Threat_Intelligence_Project/main.py (3.14.3)

File Edit Format Run Options Window Help

```python
print("Threat Intelligence Aggregator Started")

from utils.file_loader import load_file

ip_data = load_file("feeds/ip_feed.txt")
domain_data = load_file("feeds/domain_feed.txt")
url_data = load_file("feeds/url_feed.txt")

print("IP Feed Data:\n", ip_data)
print("Domain Feed Data:\n", domain_data)
print("URL Feed Data:\n", url_data)


from parsers.feed_parser import extract_ips, extract_domains, extract_urls

ips = extract_ips(ip_data)
domains = extract_domains(domain_data)
urls = extract_urls(url_data)

print("Extracted IPs:", ips)
print("Extracted Domains:", domains)
print("Extracted URLs:", urls)


from core.validator import validate_ip, validate_domain, validate_url

valid_ips = [ip for ip in ips if validate_ip(ip)]
valid_domains = [d for d in domains if validate_domain(d)]
valid_urls = [u for u in urls if validate_url(u)]

print("Valid IPs:", valid_ips)
print("Valid Domains:", valid_domains)
print("Valid URLs:", valid_urls)
```

Remove invalid entries (very important in real SOC work)

Step 8 — Normalize Data (Structured form)

Convert everything into **one standard format**

File: core/normalizer.py

normalizer.py - C:/Users/asus/OneDrive/Desktop/Threat_Intelligence_Project/core/normaliz...

File Edit Format Run Options Window Help

```python
from datetime import datetime

def normalize(indicator, ioc_type, source):

    return {
        "indicator": indicator.lower(),
        "type": ioc_type,
        "source": source,
        "timestamp": str(datetime.now())
    }
```

Update main.py

main.py - C:/Users/asus/OneDrive/Desktop/Threat_Intelligence_Project/main.py (3.14.3)

File Edit Format Run Options Window Help

```python
ips = extract_ips(ip_data)
domains = extract_domains(domain_data)
urls = extract_urls(url_data)

print("Extracted IPs:", ips)
print("Extracted Domains:", domains)
print("Extracted URLs:", urls)


from core.validator import validate_ip, validate_domain, validate_url

valid_ips = [ip for ip in ips if validate_ip(ip)]
valid_domains = [d for d in domains if validate_domain(d)]
valid_urls = [u for u in urls if validate_url(u)]

print("Valid IPs:", valid_ips)
print("Valid Domains:", valid_domains)
print("Valid URLs:", valid_urls)


from core.normalizer import normalize

ioc_database = []

# Add IPs
for ip in valid_ips:
    ioc_database.append(normalize(ip, "ip", "ip_feed"))

# Add domains
for d in valid_domains:
    ioc_database.append(normalize(d, "domain", "domain_feed"))

# Add URLs
for u in valid_urls:
    ioc_database.append(normalize(u, "url", "url_feed"))

print("IOC Database:")
for ioc in ioc_database:
    print(ioc)
```

Output

```
>>>
== RESTART: C:/Users/asus/OneDrive/Desktop/Threat_Intelligence_Project/main.py =
Threat Intelligence Aggregator Started
IP Feed Data:
 # DstIP
162.243.103.246
178.62.3.223
27.133.154.218
34.204.119.63
50.16.16.211
# END 5 entries
Domain Feed Data:
 malicious.com
phishingsite.net
badactor.org

URL Feed Data:
 http://malicious.com/malware.exe
https://phishingsite.net/login

Extracted IPs: ['162.243.103.246', '178.62.3.223', '27.133.154.218', '34.204.119.63', '50.16.16.211']
Extracted Domains: ['malicious.com', 'phishingsite.net', 'badactor.org']
Extracted URLs: ['http://malicious.com/malware.exe', 'https://phishingsite.net/login']
Valid IPs: ['162.243.103.246', '178.62.3.223', '27.133.154.218', '34.204.119.63', '50.16.16.211']
Valid Domains: ['malicious.com', 'phishingsite.net', 'badactor.org']
Valid URLs: ['http://malicious.com/malware.exe', 'https://phishingsite.net/login']
IOC Database:
{'indicator': '162.243.103.246', 'type': 'ip', 'source': 'ip_feed', 'timestamp': '2026-03-17 18:37:58.336942'}
{'indicator': '178.62.3.223', 'type': 'ip', 'source': 'ip_feed', 'timestamp': '2026-03-17 18:37:58.336972'}
{'indicator': '27.133.154.218', 'type': 'ip', 'source': 'ip_feed', 'timestamp': '2026-03-17 18:37:58.336977'}
{'indicator': '34.204.119.63', 'type': 'ip', 'source': 'ip_feed', 'timestamp': '2026-03-17 18:37:58.336980'}
{'indicator': '50.16.16.211', 'type': 'ip', 'source': 'ip_feed', 'timestamp': '2026-03-17 18:37:58.336982'}
{'indicator': 'malicious.com', 'type': 'domain', 'source': 'domain_feed', 'timestamp': '2026-03-17 18:37:58.336986'}
{'indicator': 'phishingsite.net', 'type': 'domain', 'source': 'domain_feed', 'timestamp': '2026-03-17 18:37:58.336989'}
{'indicator': 'badactor.org', 'type': 'domain', 'source': 'domain_feed', 'timestamp': '2026-03-17 18:37:58.336992'}
{'indicator': 'http://malicious.com/malware.exe', 'type': 'url', 'source': 'url_feed', 'timestamp': '2026-03-17 18:37:58.336996'}
{'indicator': 'https://phishingsite.net/login', 'type': 'url', 'source': 'url_feed', 'timestamp': '2026-03-17 18:37:58.336999'}
>>>
```

STEP 9 — Correlation Engine (MOST IMPORTANT)

File: core/correlation_engine.py Code:

correlation_engine.py - C:/Users/asus/OneDrive/Desktop/Threat_Intelligence_Project/core/c...

File Edit Format Run Options Window Help

```python
def correlate_iocs(ioc_database):

    ioc_map = {}

    # Count occurrences
    for ioc in ioc_database:

        key = ioc["indicator"]

        if key not in ioc_map:
            ioc_map[key] = {
                "type": ioc["type"],
                "sources": set([ioc["source"]])
            }
        else:
            ioc_map[key]["sources"].add(ioc["source"])

    # Convert to final list with severity
    correlated_list = []

    for indicator, data in ioc_map.items():

        count = len(data["sources"])

        # Severity logic
        if count >= 3:
            severity = "HIGH"
        elif count == 2:
            severity = "MEDIUM"
        else:
            severity = "LOW"

        correlated_list.append({
            "indicator": indicator,
            "type": data["type"],
            "sources": list(data["sources"]),
            "count": count,
            "severity": severity
        })

    return correlated_list
```

Ln: 27 Col: 0

Update main.py

```python
from core.correlation_engine import correlate_iocs

correlated_data = correlate_iocs(ioc_database)

print("\n--- Correlated IOCs ---")
for item in correlated_data:
    print(item)
```

Output

```
--- Correlated IOCs ---
{'indicator': '162.243.103.246', 'type': 'ip', 'sources': ['ip_feed'], 'count': 1, 'severity': 'LOW'}
{'indicator': '178.62.3.223', 'type': 'ip', 'sources': ['ip_feed'], 'count': 1, 'severity': 'LOW'}
{'indicator': '27.133.154.218', 'type': 'ip', 'sources': ['ip_feed'], 'count': 1, 'severity': 'LOW'}
{'indicator': '34.204.119.63', 'type': 'ip', 'sources': ['ip_feed'], 'count': 1, 'severity': 'LOW'}
{'indicator': '50.16.16.211', 'type': 'ip', 'sources': ['ip_feed'], 'count': 1, 'severity': 'LOW'}
{'indicator': 'malicious.com', 'type': 'domain', 'sources': ['domain_feed'], 'count': 1, 'severity': 'LOW'}
{'indicator': 'phishingsite.net', 'type': 'domain', 'sources': ['domain_feed'], 'count': 1, 'severity': 'LOW'}
{'indicator': 'badactor.org', 'type': 'domain', 'sources': ['domain_feed'], 'count': 1, 'severity': 'LOW'}
{'indicator': 'http://malicious.com/malware.exe', 'type': 'url', 'sources': ['url_feed'], 'count': 1, 'severity': 'LOW'}
{'indicator': 'https://phishingsite.net/login', 'type': 'url', 'sources': ['url_feed'], 'count': 1, 'severity': 'LOW'}
>>>
Ln: 52 Col: 0
```

STEP 10 — Blocklist Generator

Create files SOC tools can use to **block threats.**

File: core/blocklist_generator.py

blocklist_generator.py - C:/Users/asus/OneDrive/Desktop/Threat_Intelligence_Project/core/b...

File Edit Format Run Options Window Help

```python
def generate_blocklists(correlated_data):

    ip_list = []
    domain_list = []
    url_list = []

    for item in correlated_data:

        if item["severity"] in ["MEDIUM", "HIGH"]:

            if item["type"] == "ip":
                ip_list.append(item["indicator"])

            elif item["type"] == "domain":
                domain_list.append(item["indicator"])

            elif item["type"] == "url":
                url_list.append(item["indicator"])

    return ip_list, domain_list, url_list
```

Save Blocklists

```python
def save_blocklist(filename, data):

    with open(filename, "w") as f:
        for item in data:
            f.write(item + "\n")
```

Update main.py

```python
from core.blocklist_generator import generate_blocklists, save_blocklist

ip_blocklist, domain_blocklist, url_blocklist = generate_blocklists(correlated_data)

save_blocklist("outputs/blocklist_ips.txt", ip_blocklist)
save_blocklist("outputs/blocklist_domains.txt", domain_blocklist)
save_blocklist("outputs/blocklist_urls.txt", url_blocklist)

print("\nBlocklists generated!")
```

Ln: 77 Col: 32

STEP 11 — Report Generator

Create a **summary report (SOC-style output)**

File: core/report_generator.py

report_generator.py - C:/Users/asus/OneDrive/Desktop/Threat_Intelligence_Project/core/rep...

File Edit Format Run Options Window Help

```python
import json

def generate_report(correlated_data):

    total = len(correlated_data)

    high = len([i for i in correlated_data if i["severity"] == "HIGH"])
    medium = len([i for i in correlated_data if i["severity"] == "MEDIUM"])
    low = len([i for i in correlated_data if i["severity"] == "LOW"])

    report = {
        "total_iocs": total,
        "high_risk": high,
        "medium_risk": medium,
        "low_risk": low
    }

    return report


def save_report(report):

    with open("outputs/report.json", "w") as f:
        json.dump(report, f, indent=4)
```

Update main.py

```python
from core.report_generator import generate_report, save_report

report = generate_report(correlated_data)
save_report(report)

print("\nReport generated!")
print(report)
```

Ln: 88 Col: 13

COMPLETE PROJECT CODE:

Here is your **final main.py**

main.py - C:\Users\asus\OneDrive\Desktop\Threat_Intelligence_Project\main.py (3.14.3)

File Edit Format Run Options Window Help

```python
from utils.file_loader import load_file
from parsers.feed_parser import extract_ips, extract_domains, extract_urls
from core.validator import validate_ip, validate_domain, validate_url
from core.normalizer import normalize
from core.correlation_engine import correlate_iocs
from core.blocklist_generator import generate_blocklists, save_blocklist
from core.report_generator import generate_report, save_report

# Step 1: Load feeds
ip_data = load_file("feeds/ip_feed.txt")
domain_data = load_file("feeds/domain_feed.txt")
url_data = load_file("feeds/url_feed.txt")

# Step 2: Parse
ips = extract_ips(ip_data)
domains = extract_domains(domain_data)
urls = extract_urls(url_data)

# Step 3: Validate
valid_ips = [ip for ip in ips if validate_ip(ip)]
valid_domains = [d for d in domains if validate_domain(d)]
valid_urls = [u for u in urls if validate_url(u)]

# Step 4: Normalize
ioc_database = []

for ip in valid_ips:
    ioc_database.append(normalize(ip, "ip", "ip_feed"))

for d in valid_domains:
    ioc_database.append(normalize(d, "domain", "domain_feed"))

for u in valid_urls:
    ioc_database.append(normalize(u, "url", "url_feed"))

# Step 5: Correlate
correlated_data = correlate_iocs(ioc_database)

# Step 6: Blocklists
ip_blocklist, domain_blocklist, url_blocklist = generate_blocklists(correlated_data)

save_blocklist("outputs/blocklist_ips.txt", ip_blocklist)
save_blocklist("outputs/blocklist_domains.txt", domain_blocklist)
save_blocklist("outputs/blocklist_urls.txt", url_blocklist)

# Step 7: Report
report = generate_report(correlated_data)
save_report(report)

print("\nProject Completed Successfully!")
```

Ln: 51 Col: 0

UPDATE 1 — Add HASH + EMAIL Detection

Update: parsers/feed_parser.py

Add this:

```
def extract_hashes(text):
    import re

    md5_pattern = r"\b[a-fA-F0-9]{32}\b"
    sha256_pattern = r"\b[a-fA-F0-9]{64}\b"

    md5 = re.findall(md5_pattern, text)
    sha256 = re.findall(sha256_pattern, text)

    return md5 + sha256


def extract_emails(text):
    import re

    pattern = r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"
    return re.findall(pattern, text)
```

Update: main.py

Add:

```
from parsers.feed_parser import extract_hashes, extract_emails

hashes = extract_hashes(ip_data + domain_data + url_data)
emails = extract_emails(ip_data + domain_data + url_data)
```

Ln: 57 Col: 0

Add Validation (core/validator.py)

```
def validate_hash(h):
    return len(h) in [32, 64]

def validate_email(e):
    return "@" in e
```

Add to normalization

```python
for h in hashes:
    if validate_hash(h):
        ioc_database.append(normalize(h, "hash", "feed"))

for e in emails:
    if validate_email(e):
        ioc_database.append(normalize(e, "email", "feed"))
```

UPDATE 2 — Add LIVE Threat Feed

Update: utils/file_loader.py

Add:

```python
import requests

def load_url(url):
    try:
        response = requests.get(url)
        return response.text
    except:
        return ""
```

Update: main.py

Add:

```python
live_feed = load_url("https://feodotracker.abuse.ch/downloads/ipblocklist.txt")
```

Ln: 59 Col: 79

Then include it in parsing:

```python
ips += extract_ips(live_feed)
```

UPGRADE 3 — Advanced Reporting

Update: core/report_generator.py

Replace with:

report_generator.py - C:\Users\asus\OneDrive\Desktop\Threat_Intelligence_Project\core\report_gen...

File Edit Format Run Options Window Help

```python
import json

def generate_report(correlated_data):

    total = len(correlated_data)

    high = [i for i in correlated_data if i["severity"] == "HIGH"]
    medium = [i for i in correlated_data if i["severity"] == "MEDIUM"]
    low = [i for i in correlated_data if i["severity"] == "LOW"]

    # Top repeated IOCs
    top_iocs = sorted(correlated_data, key=lambda x: x["count"], reverse=True)[:5]

    report = {
        "total_iocs": total,
        "high_risk": len(high),
        "medium_risk": len(medium),
        "low_risk": len(low),
        "top_threats": top_iocs
    }

    return report


def save_report(report):
    with open("outputs/report.json", "w") as f:
        json.dump(report, f, indent=4)
```

OUTPUT:

{} report.json M

C: > Users > asus > OneDrive > Desktop > Threat_Intelligence_Project > outputs > {} report


```json
{
    "total_iocs": 10,
    "high_risk": 0,
    "medium_risk": 0,
    "low_risk": 10,
    "top_threats": [
        {
            "indicator": "162.243.103.246",
            "type": "ip",
            "sources": [
                "ip_feed"
            ],
            "count": 1,
            "severity": "LOW"
        },
        {
            "indicator": "178.62.3.223",
            "type": "ip",
            "sources": [
                "ip_feed"
            ],
            "count": 1,
            "severity": "LOW"
        },
        {
            "indicator": "27.133.154.218",
            "type": "ip",
            "sources": [
                "ip_feed"
            ],
            "count": 1,
            "severity": "LOW"
        },
        {
            "indicator": "34.204.119.63",
            "type": "ip",
            "sources": [
```


Add Logging (in main)

In main.py:

```python
print("[INFO] Loading feeds...")
print("[INFO] Parsing IOCs...")
print("[INFO] Generating report...")
```

Ln: 67 Col: 36

OUTPUT:

```
== RESTART: C:\Users\asus\OneDrive\Desktop\Threat_Intelligence_Project\main.py =

Project Completed Successfully!
[INFO] Loading feeds...
[INFO] Parsing IOCs...
[INFO] Generating report...
>>>
```